

Chardi.ai — Metrics Summary

Scraping coverage, data freshness, and regional breakdown across 10 Indian government procurement portals

portals scraped

10

state + central gov

backfill window

30d

on first deploy

daily refresh

2 AM

IST cron, railway

states covered

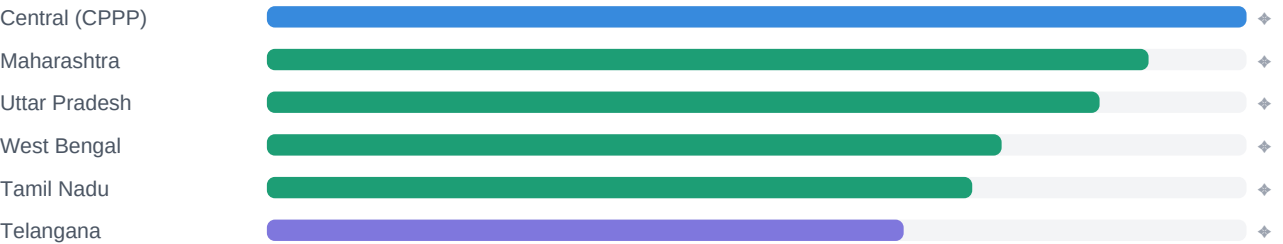
9

+ central (CPPP)

PORTAL COVERAGE — SOURCE & SCRAPING METHOD

CPPP — Central Public Procurement Portal eprocure.gov.in · HTML table parsing via BeautifulSoup · paginated /cpppdata endpoint · SHA-256 deduplication	HTML
AP eProcurement — Andhra Pradesh ap.eprocurement.gov.in · structured JSON API · fields mapped from tender_number, closing_date	JSON
KPPP — Karnataka eProcurement kppp.karnataka.gov.in · REST POST API · 3 separate endpoints for Goods, Works, Services categories	REST API
Telangana eProcurement tender.telangana.gov.in · DataTables JSON API · sorted by published_date desc · session bootstrap required	JSON
Maharashtra eProcurement mahatenders.gov.in · Playwright browser automation · CAPTCHA solved via local OCR server	CAPTCH A
Tamil Nadu eProcurement tntenders.gov.in · Playwright browser automation · CAPTCHA solved via local OCR server	CAPTCH A
UP eProcurement — Uttar Pradesh etender.up.nic.in · Playwright browser automation · CAPTCHA solved via local OCR server	CAPTCH A
WB Tenders — West Bengal wbtenders.gov.in · Playwright browser automation · CAPTCHA solved via local OCR server	CAPTCH A
Rajasthan eProcurement sppp.rajasthan.gov.in · Playwright browser automation · CAPTCHA solved via local OCR server	CAPTCH A
Kerala Tenders etenders.kerala.gov.in · Playwright browser automation · CAPTCHA solved via local OCR server	CAPTCH A

REGIONAL COVERAGE





Bar width = relative volume estimate. Blue = central, green = high-volume states, purple = mid-volume states.

DATA FRESHNESS STRATEGY

● Initial backfill On first deployment, all scrapers pull 30 days of historical data. Each portal is paginated until the cutoff date is reached, then stops. Records are upserted — safe to re-run.	● Daily cron — 2:00 AM IST Railway cron runs main.py every night. Each scraper uses a 1-day window (<code>BACKFILL_DAYS = 1</code>), fetching only new tenders from the previous 24 hours.
● Idempotent upserts Every insert uses <code>ON CONFLICT (id) DO UPDATE</code>. IDs are SHA-256 hashes of <code>portal::tender_ref_no</code>, so duplicate runs never create duplicates — they update status and deadline.	● Chronological cutoff Scrapers paginate newest-first and stop as soon as a row's <code>published_at</code> falls before the cutoff. No wasted requests fetching old data after the threshold is hit.

OCR SERVER — SELF-HOSTED CAPTCHA SOLVER

ddddocr — deployed free on Railway

6 of the 10 portals (Maharashtra, Tamil Nadu, Uttar Pradesh, West Bengal, Rajasthan, Kerala) block scraping with image CAPTCHAs. Commercial solvers like 2captcha and Anti-Captcha charge per-solve (~\$1–3 per 1,000), which adds up at scale. Instead, a lightweight FastAPI microservice runs **ddddocr** — an open-source neural network optimised for alphanumeric government-portal CAPTCHAs. It's deployed as a separate Railway service alongside the scrapers.

The `unified_Scrapper.py` POSTs the raw CAPTCHA image bytes to `POST /v1/decode-captcha`. The server strips whitespace artifacts, returns the cleaned prediction, and the scraper injects it into the form submission. No external API keys, no per-request costs.

Tech stack

- PostgreSQL via Supabase (DB)
- FastAPI + psycpg2 (backend)
- Playwright + requests (scrapers)
- dddocr + FastAPI (OCR server)
- HTML / CSS / JS (dashboard)
- Railway (backend + scrapers + OCR)
- Netlify (frontend hosting)

AI usage

- Claude + Gemini — scraper logic, API reverse-engineering, CAPTCHA bypass flows
 - Claude — dashboard UI (filters, charts, export, pagination)
- All generated code reviewed and adapted to handle portal-specific quirks.
- [View live dashboard → indian-tenderdashboard.netlify.app](#)